Assignment 3

Task 1: Implementing MiniMax Algorithm

```
_best_move(depth):
    moves = _instance.valid_moves()

    top_score = MIN_VALUE
    top_move

    /*
     * lambda for applying a move and evaluating the resulting
     * instance
     */
    search_node = [&](instance, pos, special) {
        instance.apply_move(pos, special)

        /// Commence recursion
        score = _search_minimax(instance, depth - 1)

        /// Update top move
        if score > top_score:
            top_score = score
            top_move = move(_me, pos, _phase, special)
    }
```

```
/// Build initial game tree layer
for each pos in moves:
    /*
     * in case of choice, all possible player choices need
     * to be explored
     */
    if pos is choice:
        for all player in instance.player_count():
            search_node(instance, pos, player)
    /*
     * in case of bonus, both bomb and override stone need
     * to be explored
     */
    else if pos is bonus:
        for types of bonus:
            search_node(instance, pos, type)
    else search_node(instance, pos, none)

return top_move
```

Task 1: Implementing MiniMax Algorithm

```
_search_minimax(instance, depth):
    if depth is 0: return evaluate(instance)

    max = instance._turn == _me;

    eval = max ? MIN_VALUE : MIN_VALUE

    eval_node = [&](instance, pos, special) {
        instance.apply_move(pos, special)

        child_eval = _search_minimax(instance, depth - 1)

        eval = max ? (eval > child_eval ? eval : child_eval)
                   : (eval > child_eval ? child_eval : eval)
    }
```

```
/// Recursively execute _search_minimax for all legal moves
for each pos in moves:
    /*
     * in case of choice, all possible player choices need
     * to be explored
     */
    if pos is choice:
        for all player in instance.player_count():
            eval_node(instance, pos, player)
    /*
     * in case of bonus, both bomb and override stone need
     * to be explored
     */
    else if pos is bonus:
        for types of bonus:
            eval_node(instance, pos, type)
    else eval_node(instance, pos, none)

return eval
```

Task 2: Implementing Alpha-Beta Pruning

```
_search_minimax_alpha_beta_pruning(instance, depth, alpha, beta):
    // ...

    /// Returns true on alpha or beta cutoff
    eval_node = [&](instance, pos, special) -> bool {
        instance.apply_move(pos, special)

        child_eval = _search_minimax_alpha_beta_pruning(instance,
           depth - 1)

        eval = max ? (eval > child_eval ? eval : child_eval)
                   : (eval > child_eval ? child_eval : eval)

        if (max) {
            if (eval >= beta) return 1;
            alpha = eval > alpha ? eval : alpha;
        }
        else {
            if (eval <= alpha) return 1;
            beta = eval > beta ? beta : eval;
        }

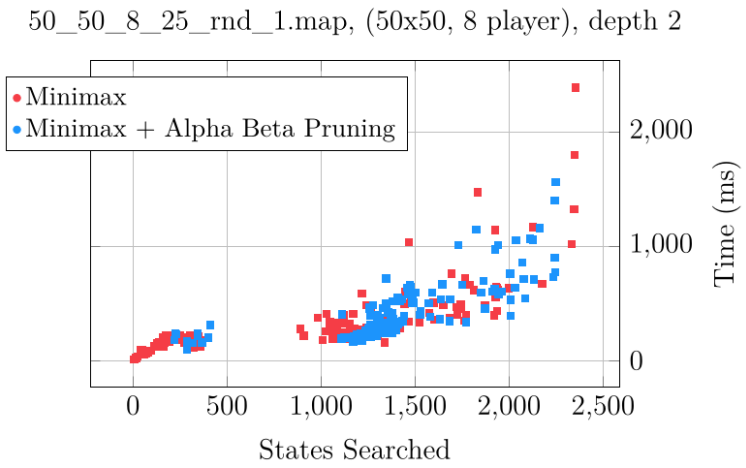
        return 0;
     }
```

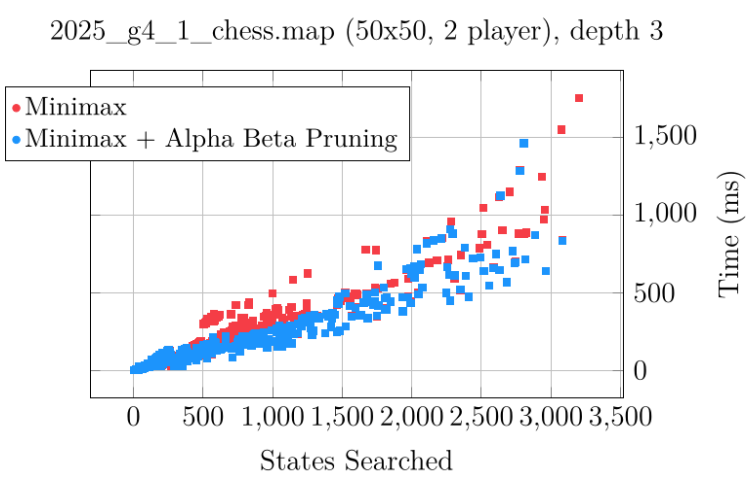
```
/// Used to break out of outer loop on alpha or beta cutoff
bool cutoff = false;

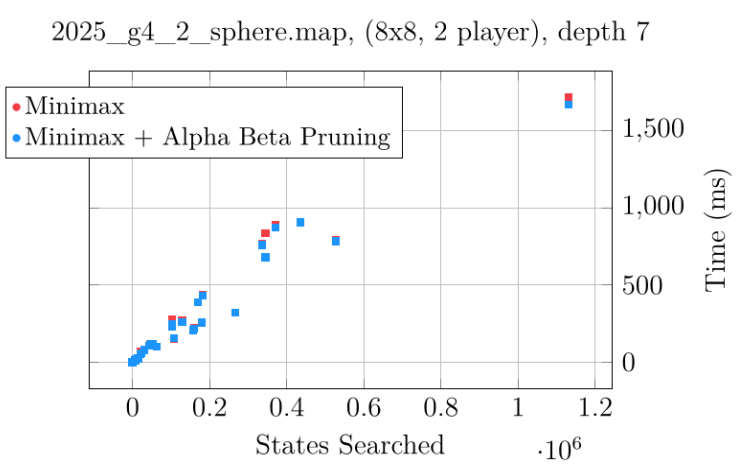
/// Recursively execute _search_minimax for all legal moves
for each pos in moves:
    /*
     * in case of choice, all possible player choices need
     * to be explored
     */
    if pos is choice:
        for all player in instance.player_count():
            if (eval_node(instance, pos, player)) {
                cutoff = true
                break
            }
    /*
     * in case of bonus, both bomb and override stone need
     * to be explored
     */
    else if pos is bonus:
        for types of bonus:
            if (eval_node(instance, pos, type)) {
                cutoff = true
                break
            }
    else if(eval_node(instance, pos, none)) {
        cutoff = true
        break
    }

return eval
```

Task 3: Efficiency of the algorithms





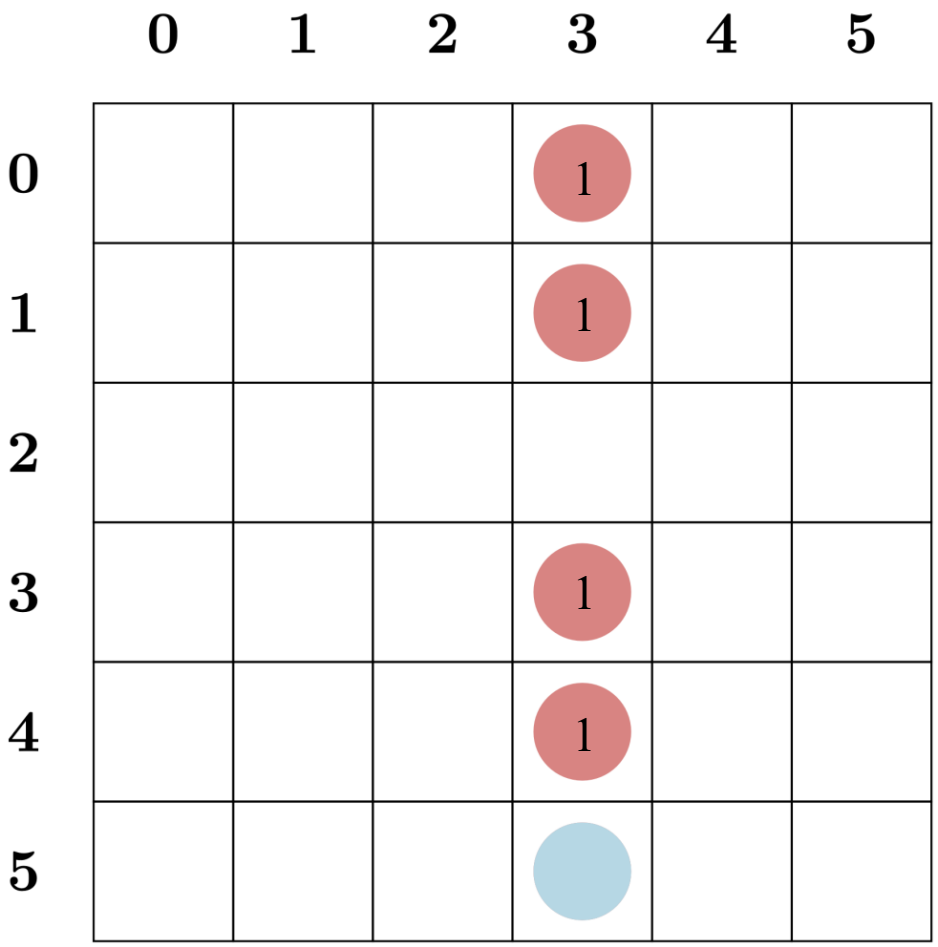


Kogge-Stone

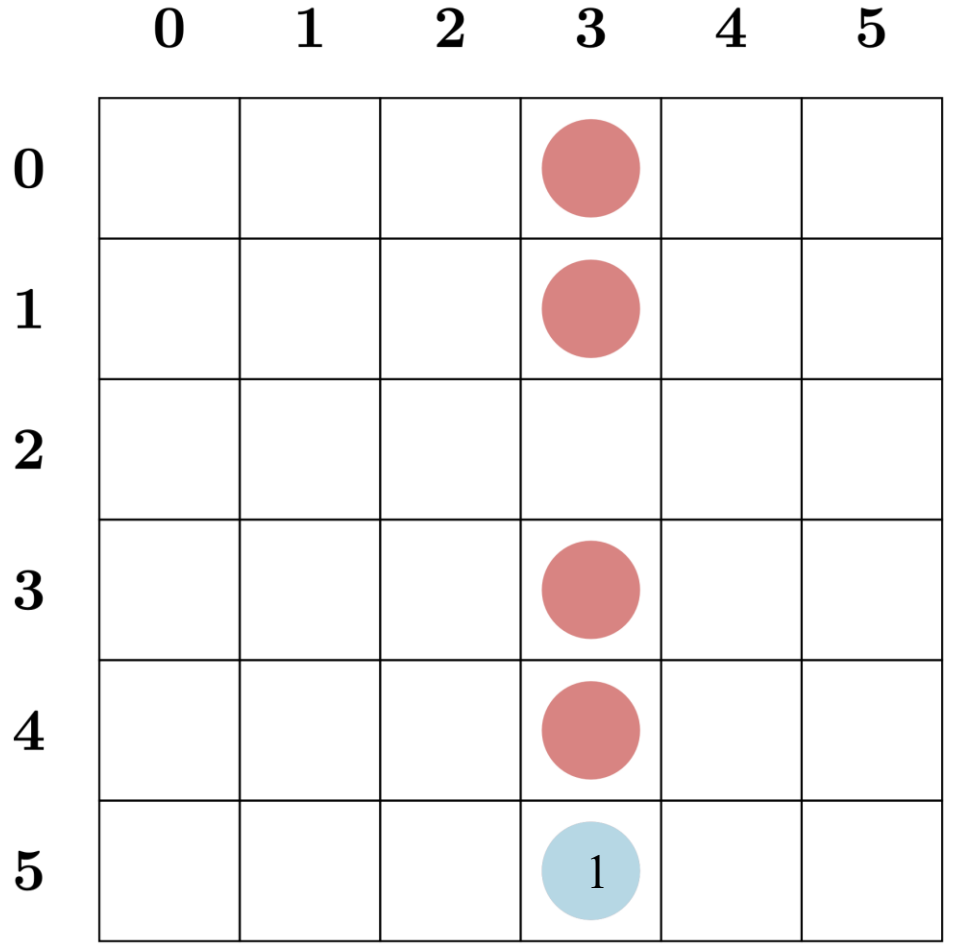
- Fast algorithm for finding valid moves
- For each direction, calculates all valid moves at once
- Logaritmic time complexity due to doubling propagation distance at each step

```
1    // propagate in the direction as long as possible
2    for(pos_t dist = 1; pro && dist < limit; dist <<= 1)
3    {
4        gen |= pro & shift(gen, dir, dist);
5        pro &= shift(pro, dir, dist);
6    }
```

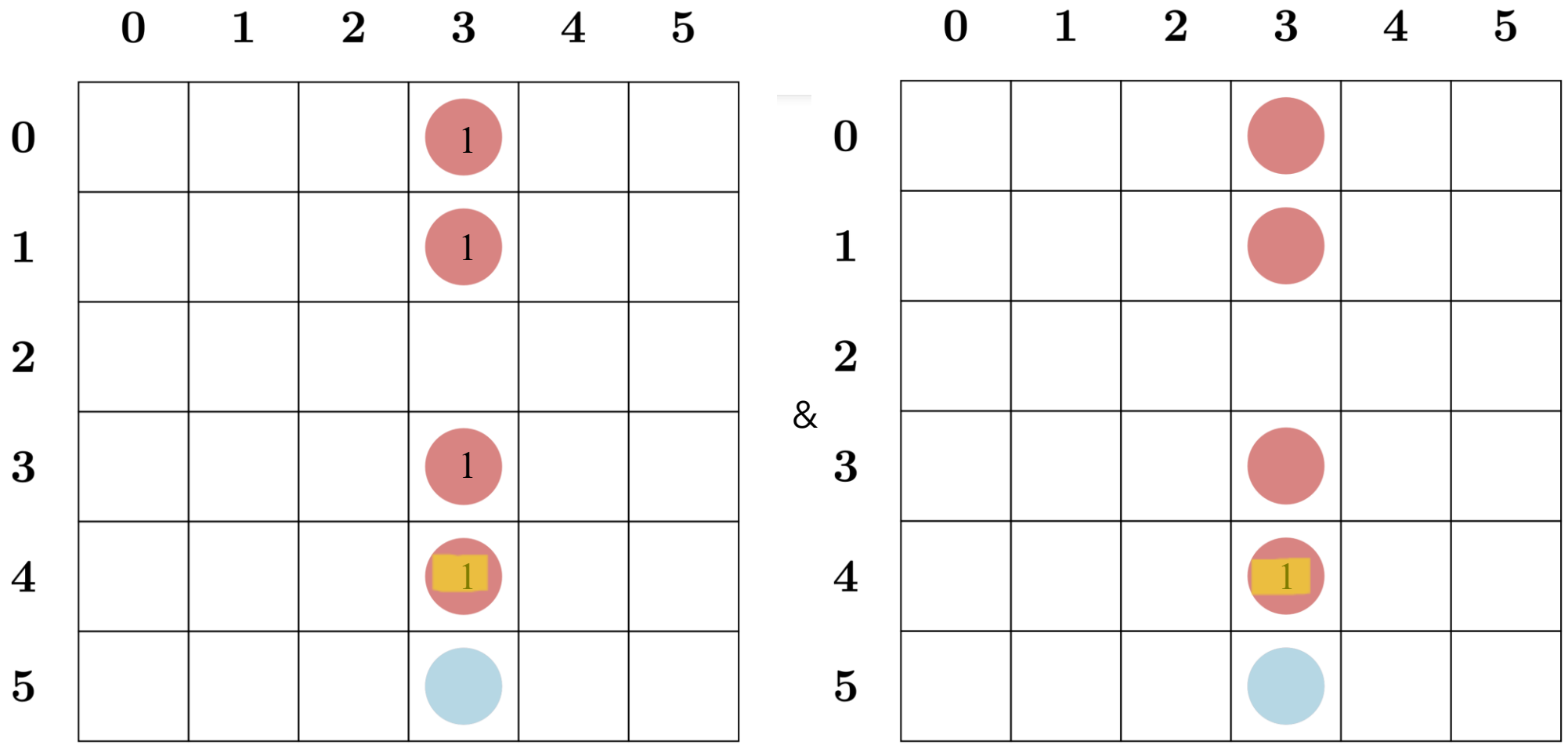
pro := o



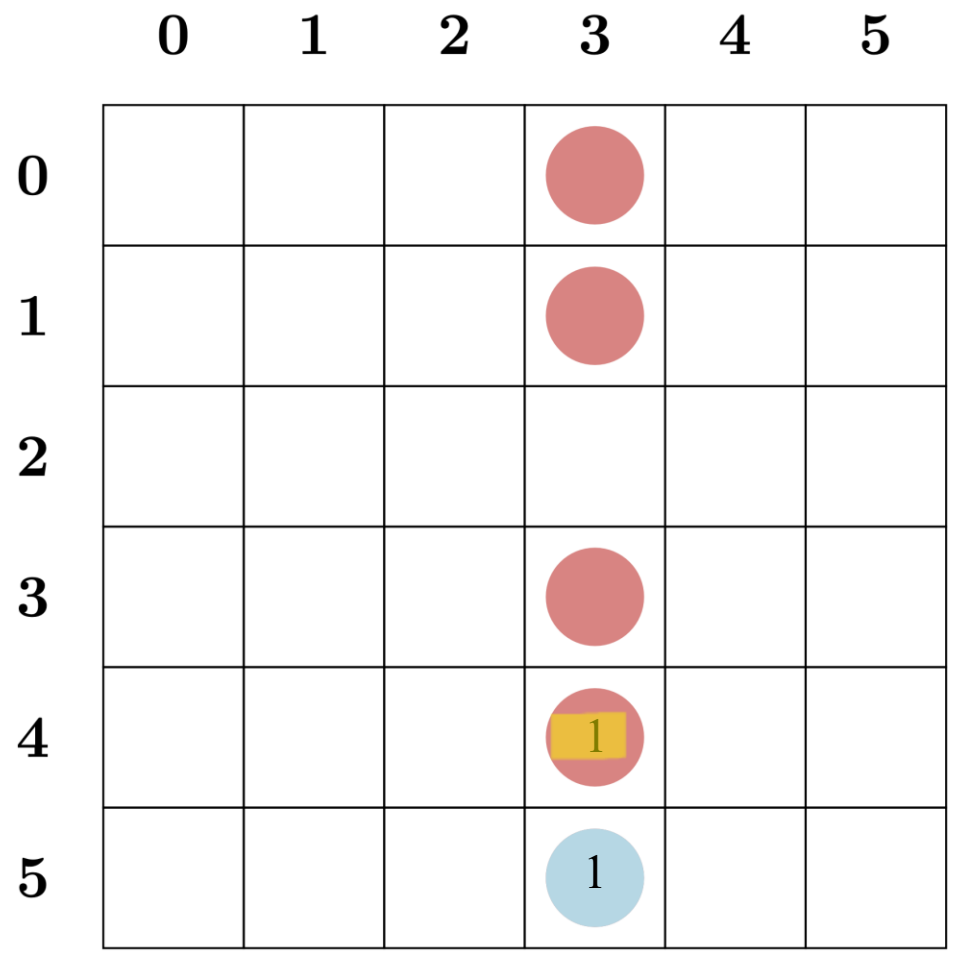
gen := p



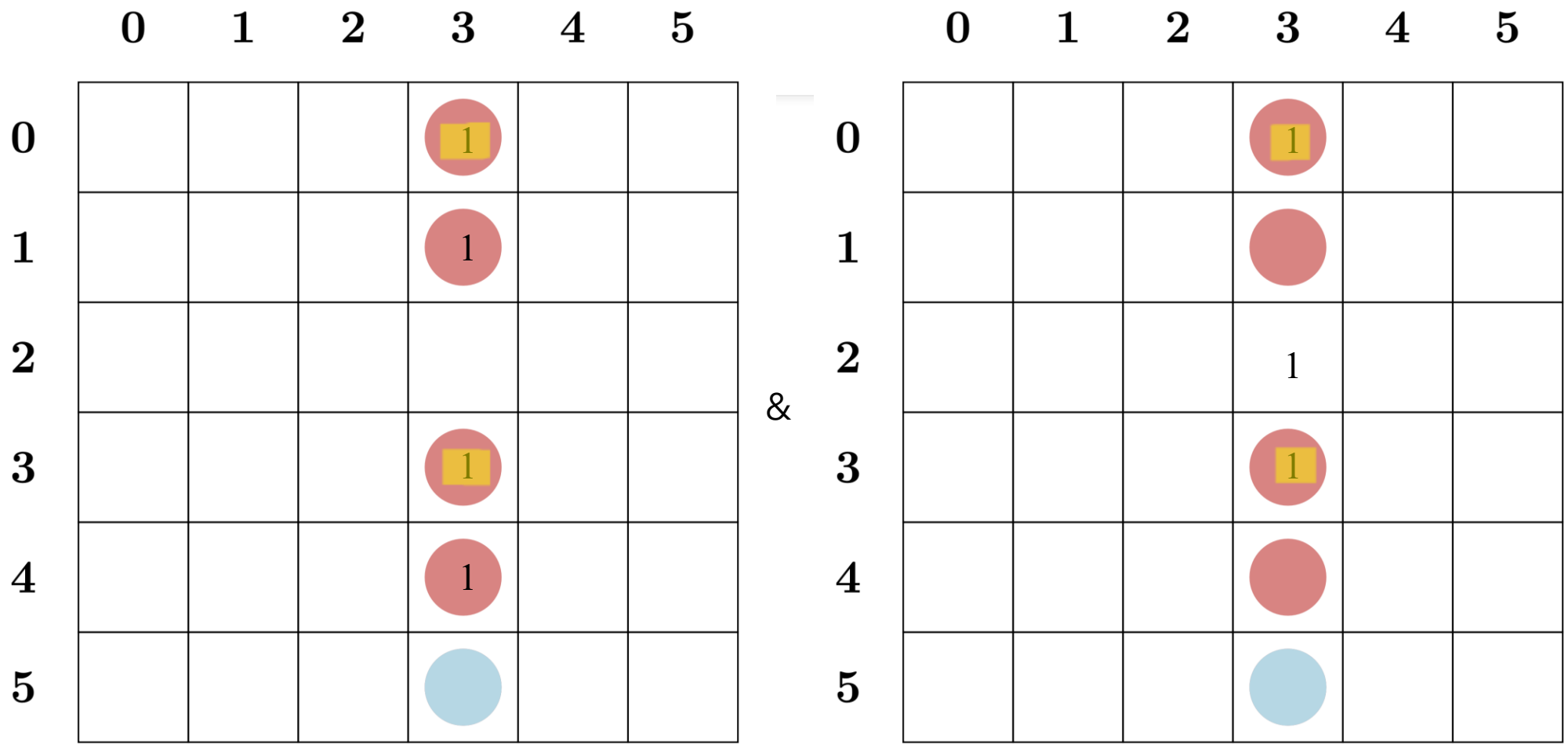
gen |= pro & shift(gen, 1)

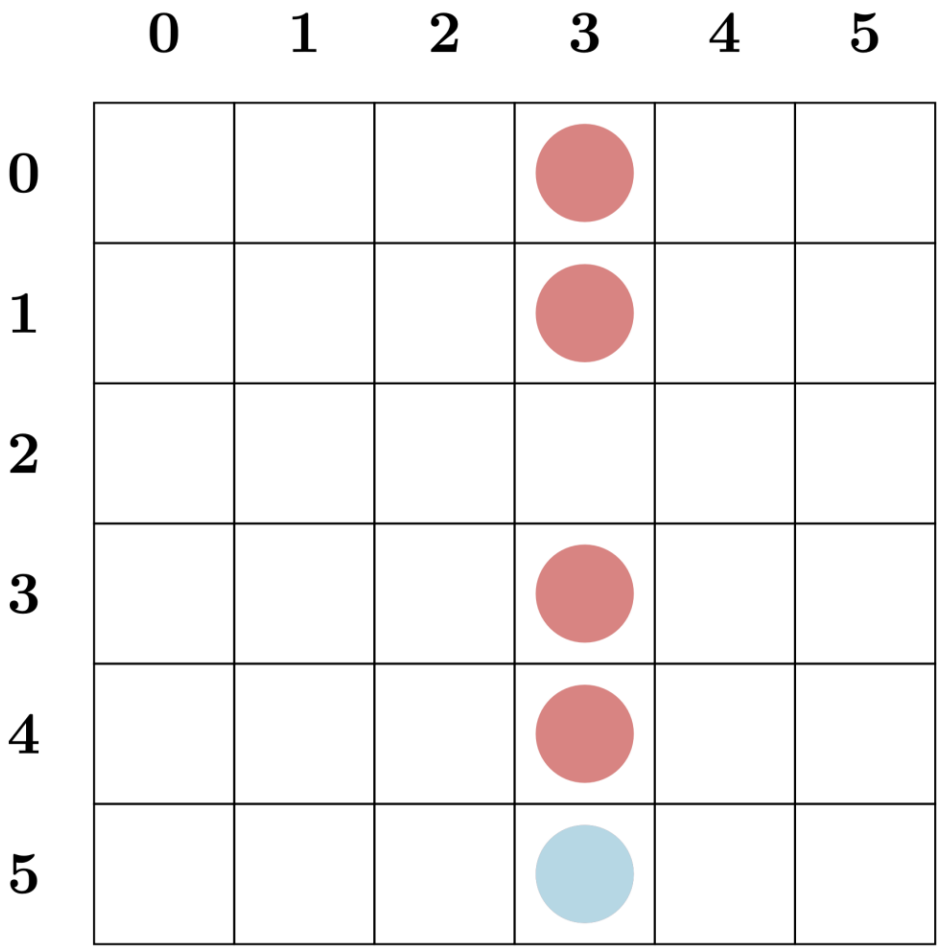


gen



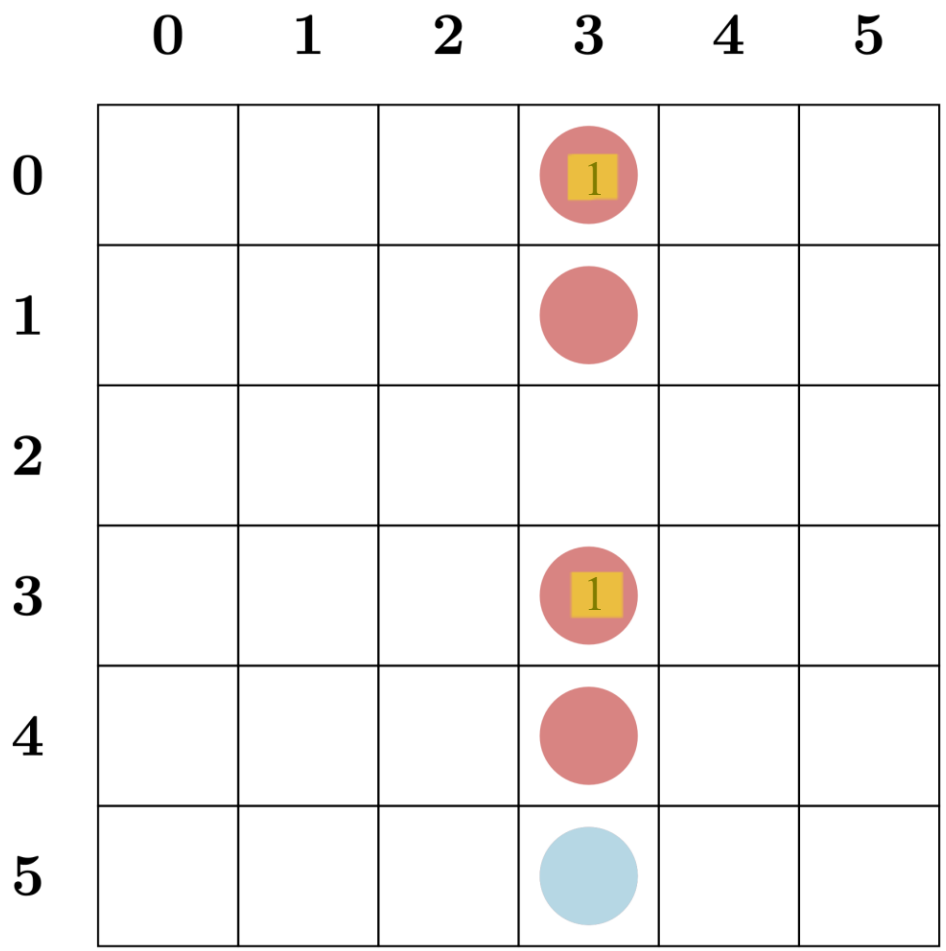
pro = pro & shift(pro, 1)



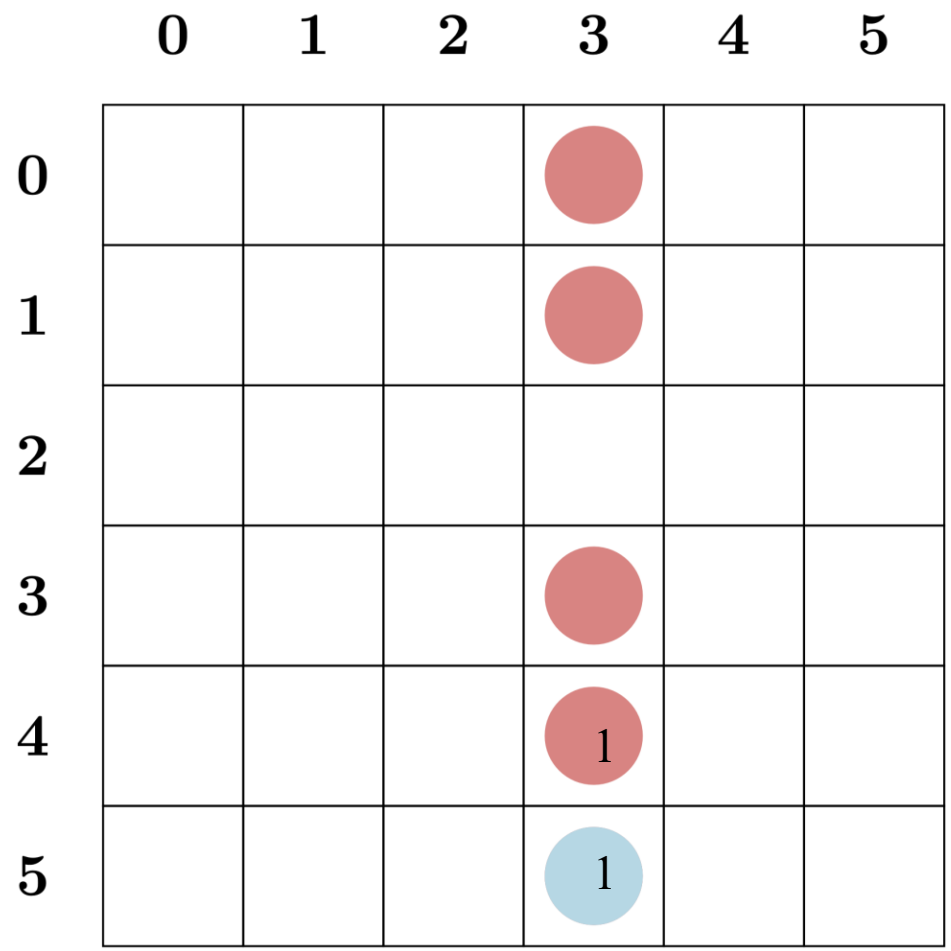


We define shift(layer, step) :=
shift(“N”, layer, step)

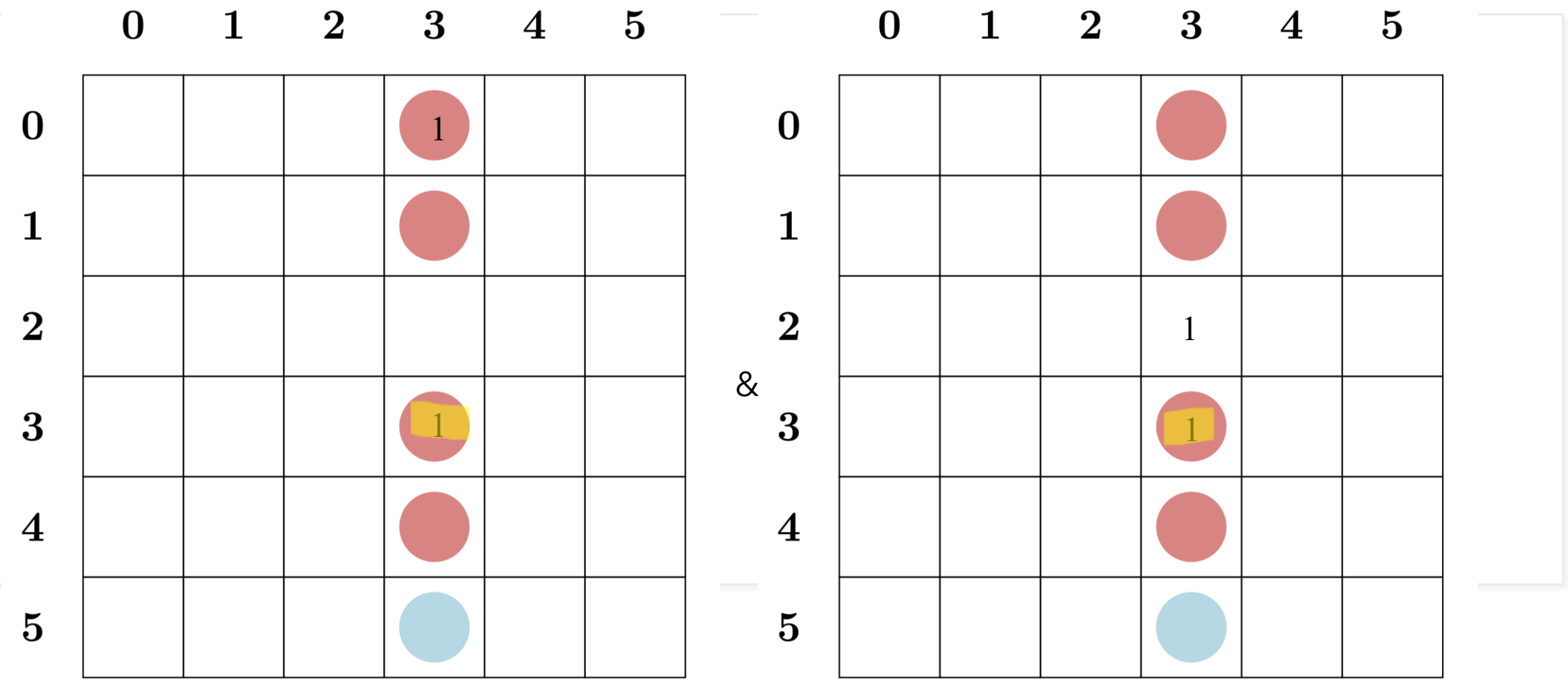
pro



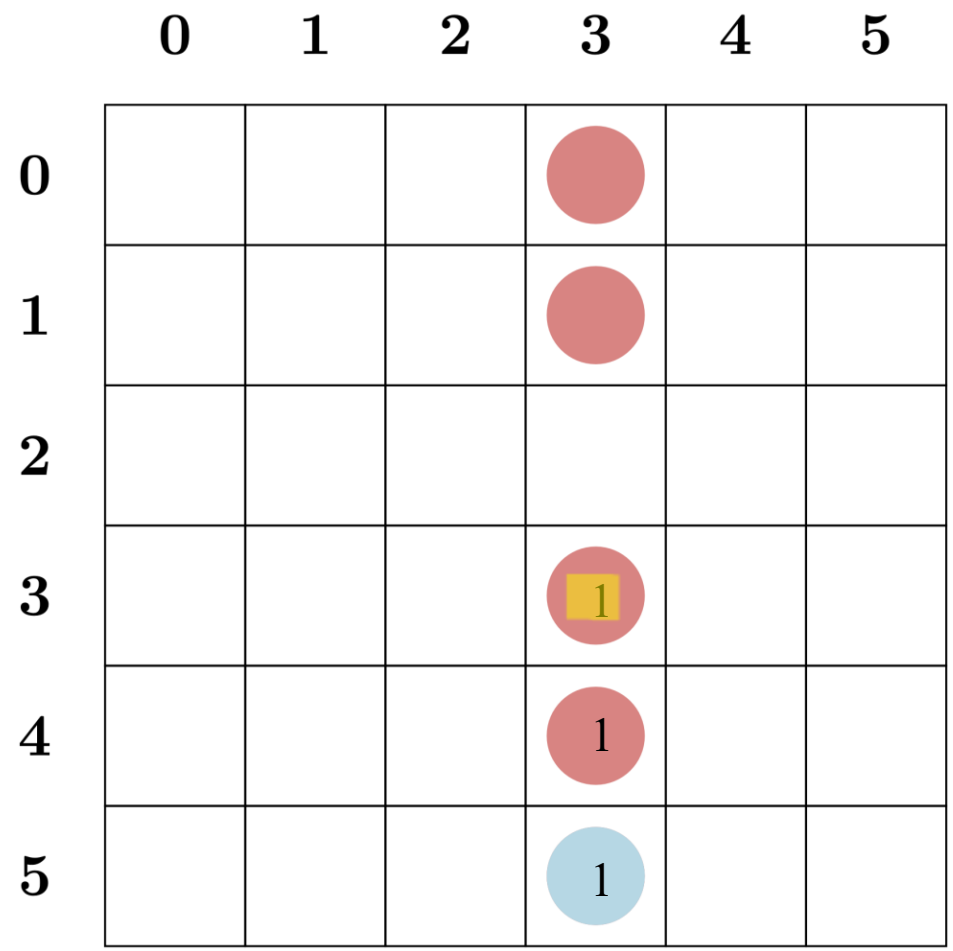
gen



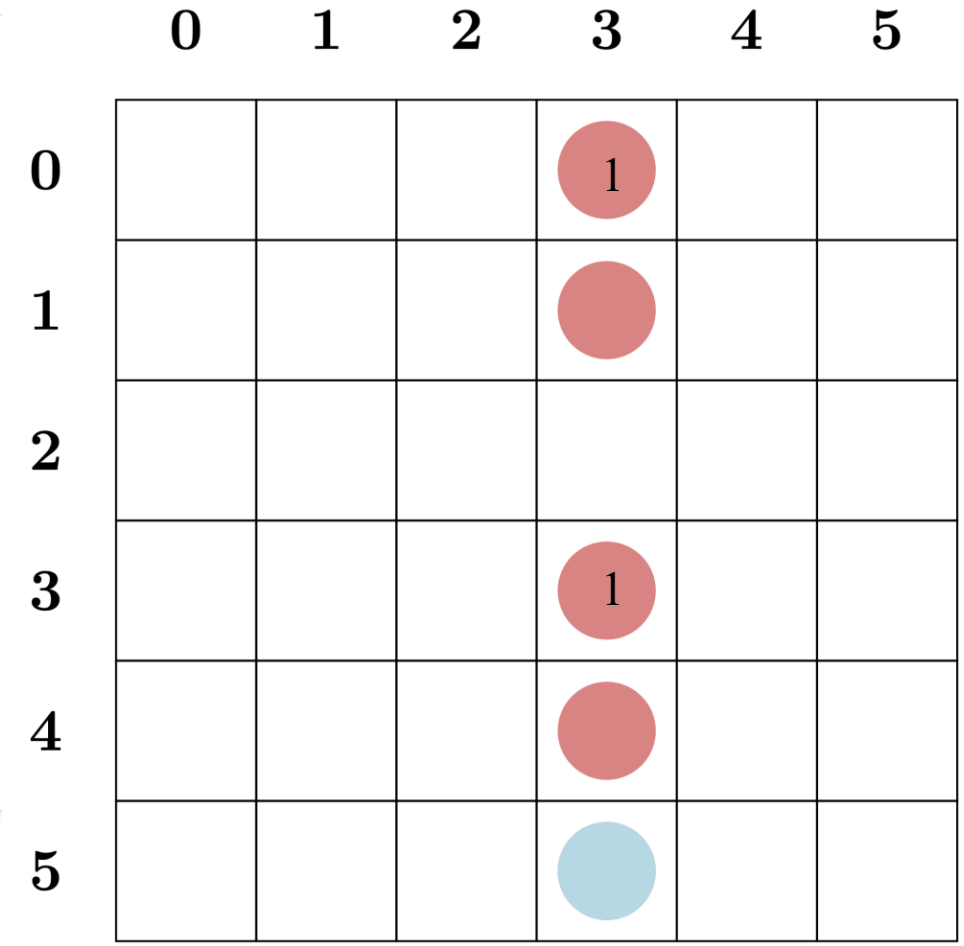
gen |= pro & shift(gen, 2)



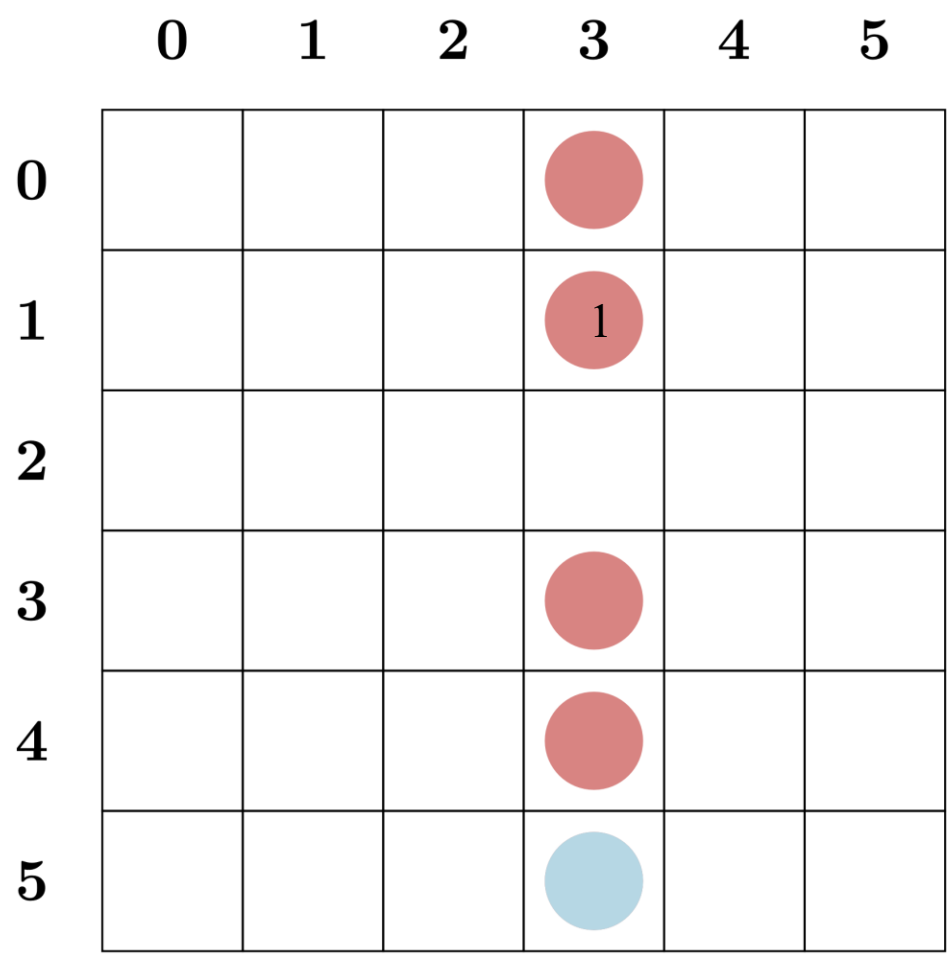
gen



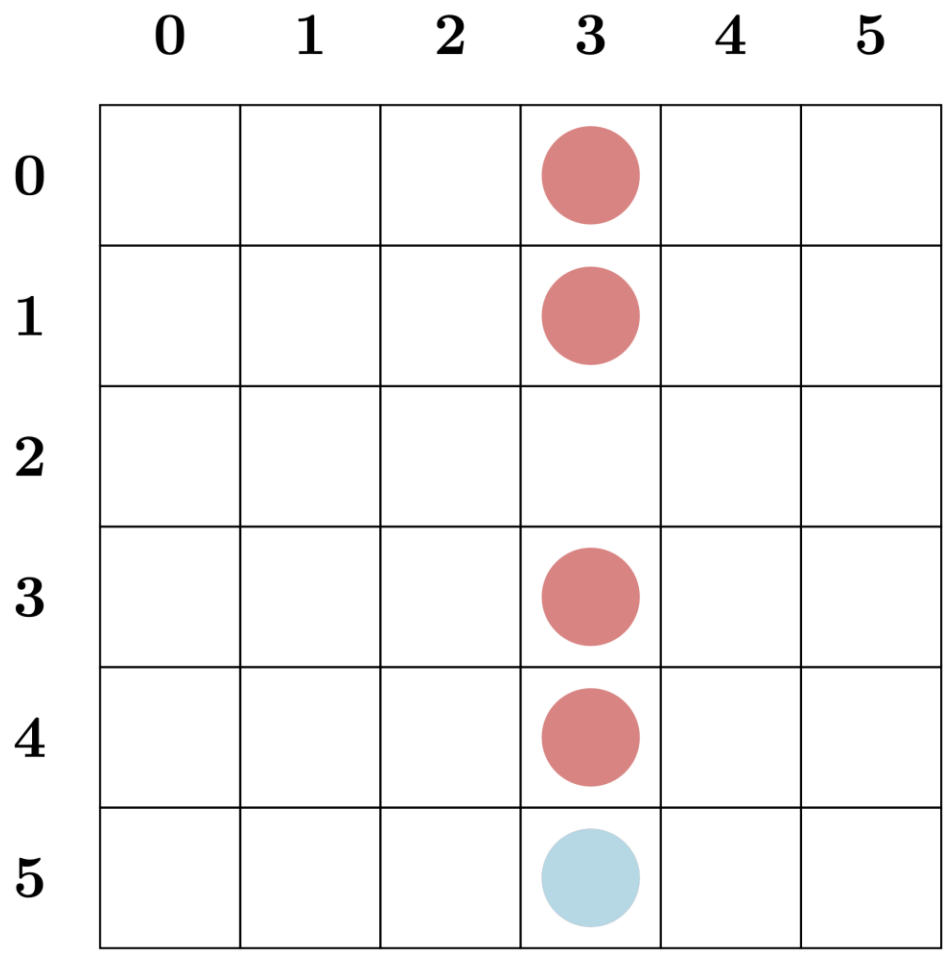
pro = pro & shift(pro, 2)



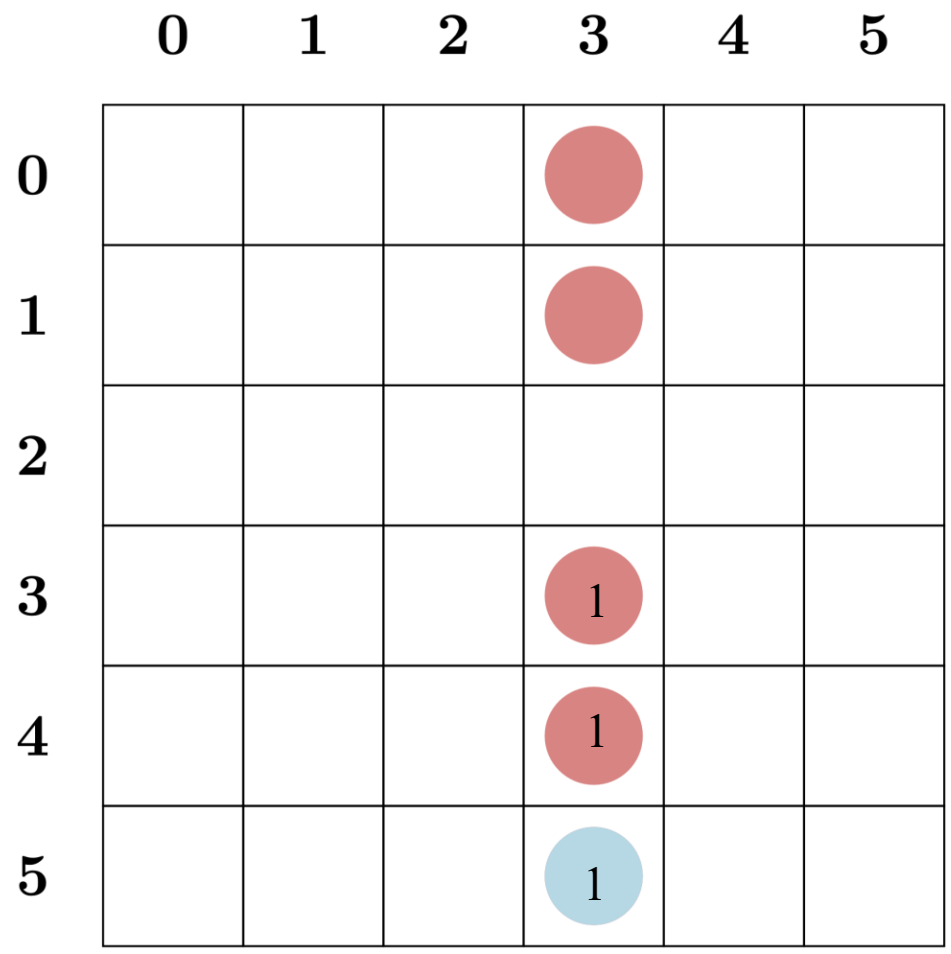
&



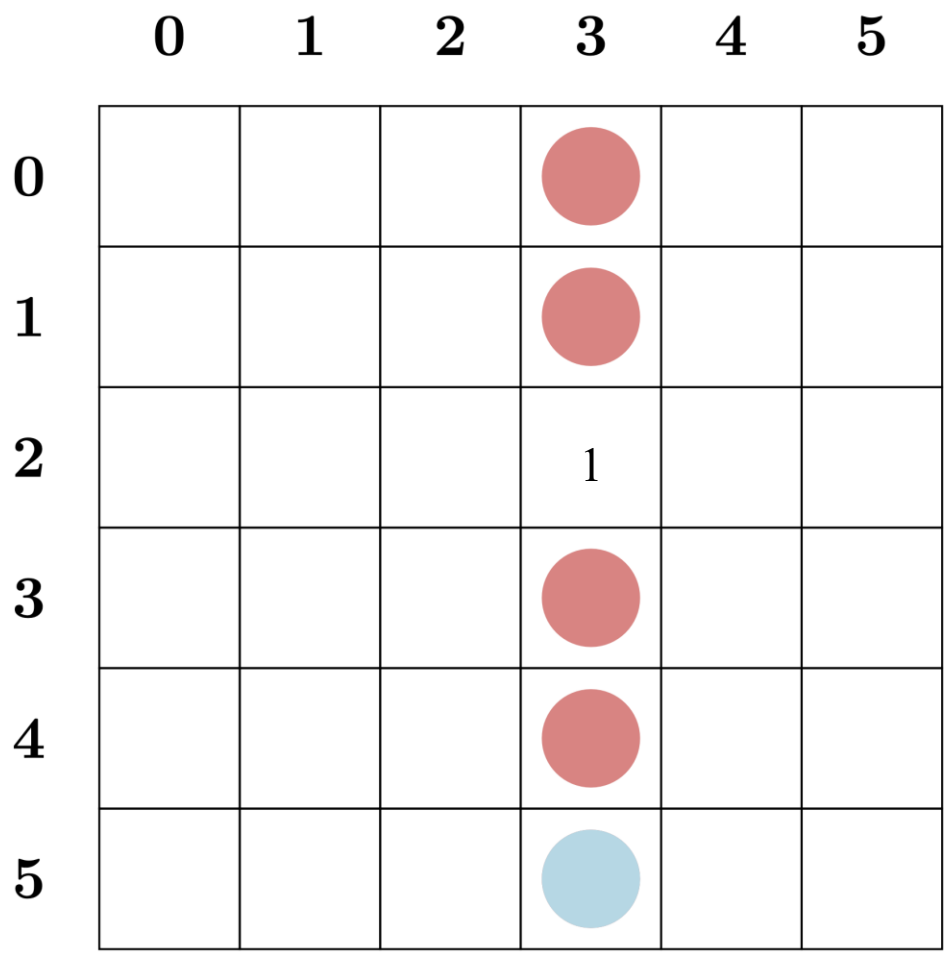
pro



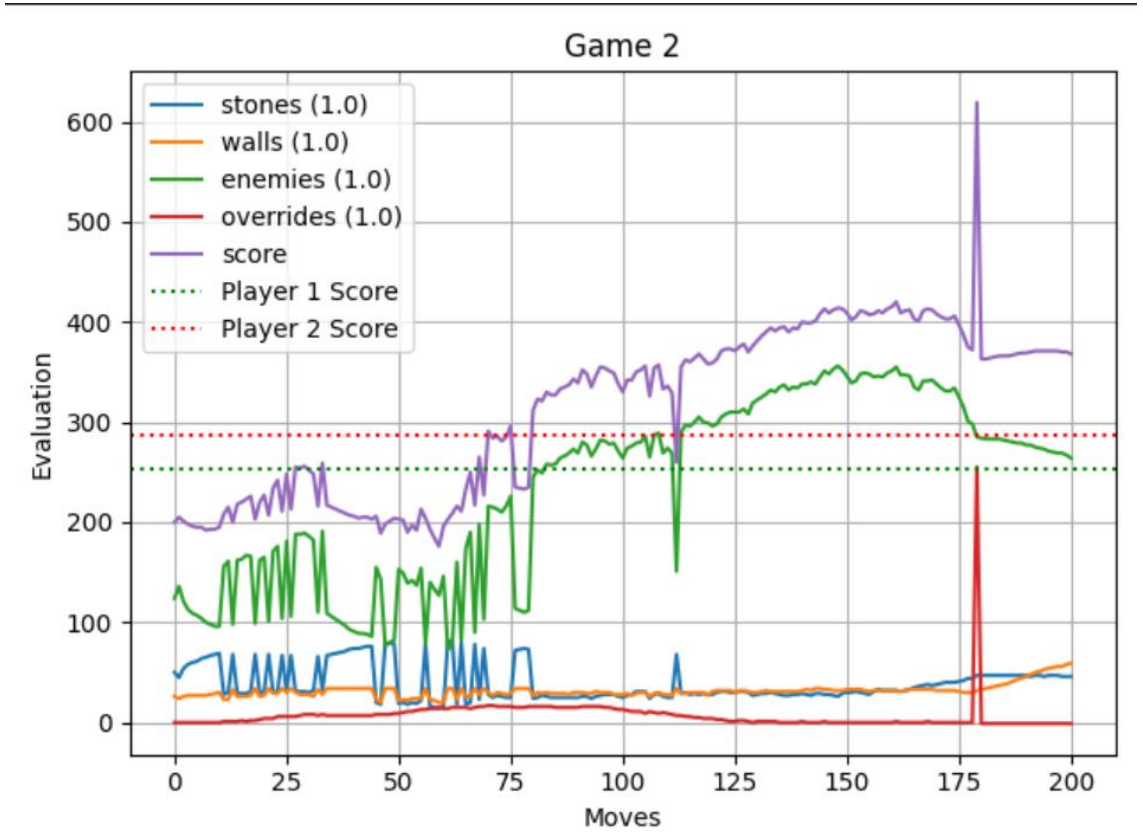
gen



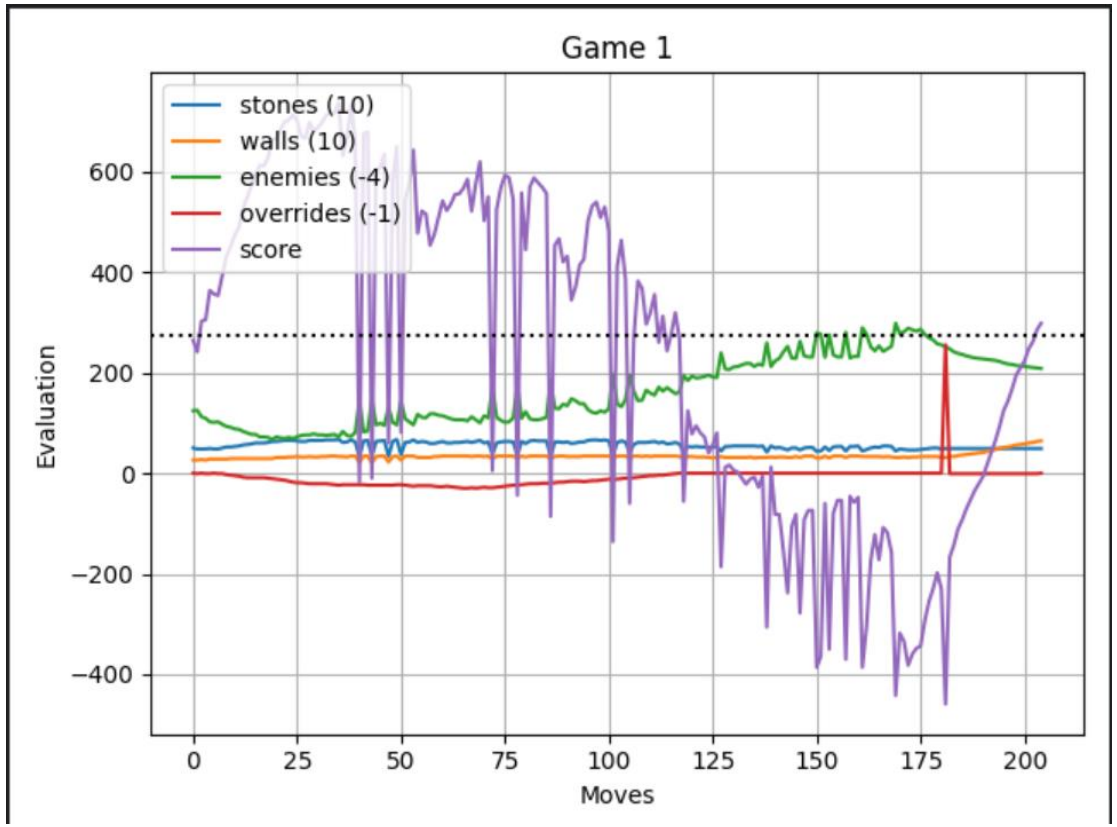
Final step:
res_place |= shift(gen & o , 1) & empty_layer



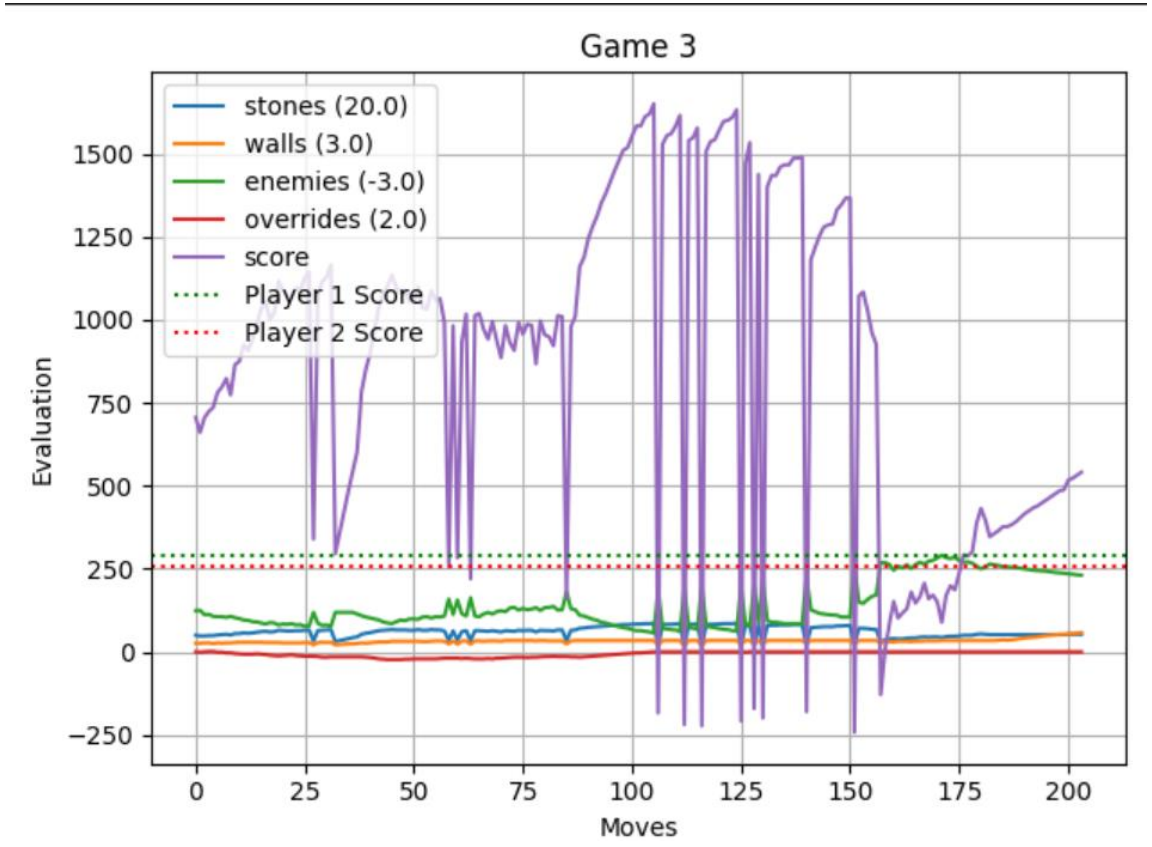
Heuristic Evaluation



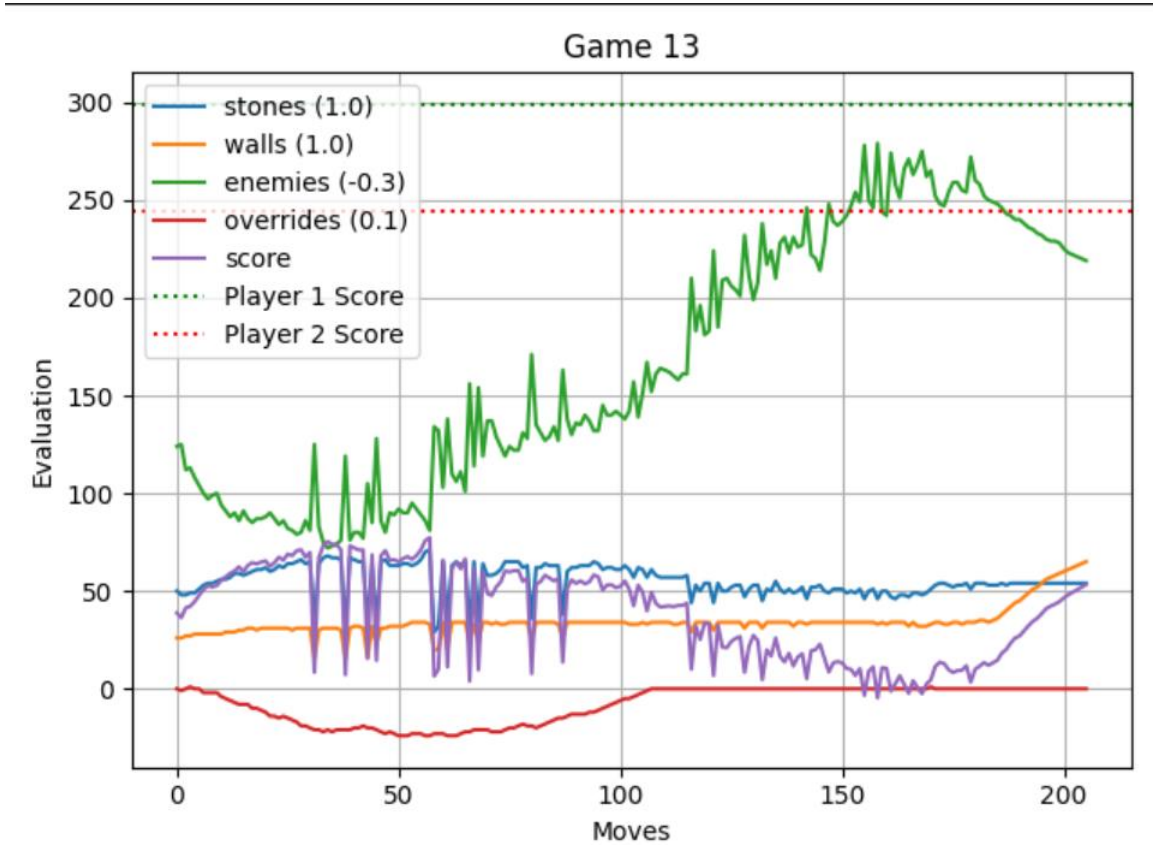
Heuristic Evaluation



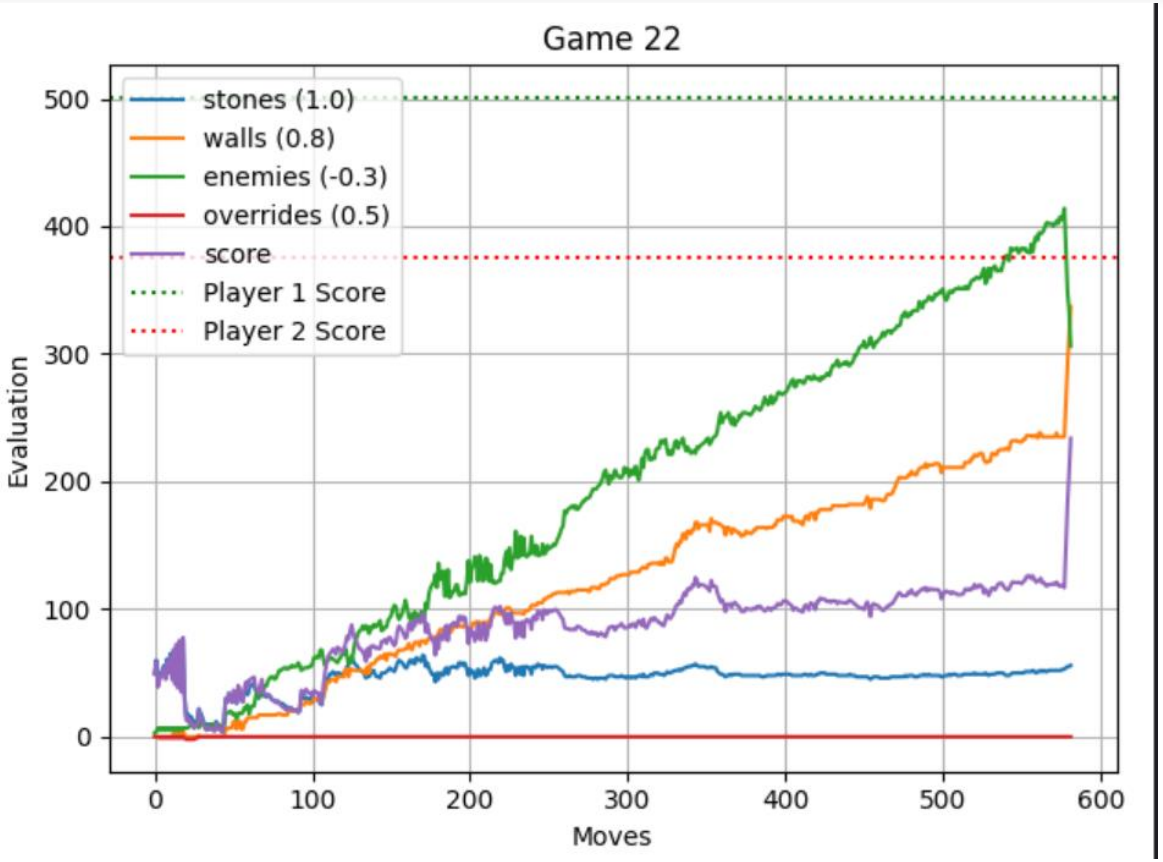
Heuristic Evaluation



Heuristic Evaluation



Heuristic Evaluation



Heuristic Evaluation – Future Goals

- Competition among our clients
- Using Matchpoint to check scores
- Using more maps and running more games
- Create a new 5th heuristic

MCTS – Monte Carlo Tree Search

1. Exploring promising areas
2. Creating new nodes

- Less dependent on heuristics
- Better *anytime behaviour* (than minimax)

IDS – Iterative Deepening Search

- Enables us to explore only certain parts of tree

- Repeatedly performing DFS with increasing depth limit

→ behaves like BFS with memory usage of DFS